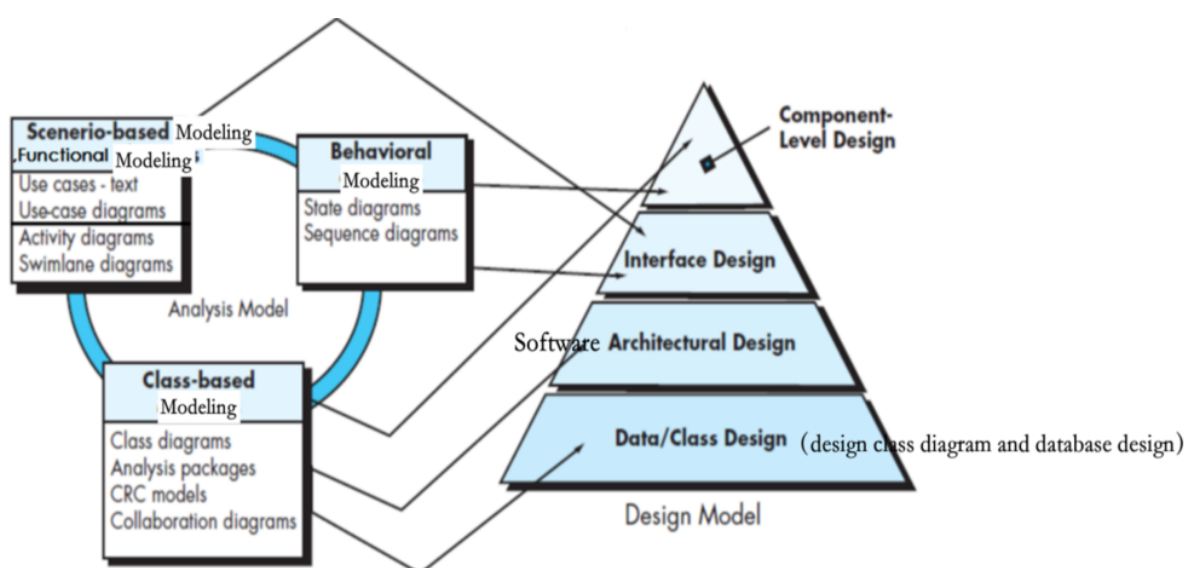


Chapter 12 设计概念 Design Concepts

软件设计包括实现高质量系统或产品的开发的一组**原则principles**，**概念concepts**和**实践practices**。

设计阶段 from bottom to the top

- Data/class diagram:数据设计/类图设计（分析类图细化为设计类图）
- Software Architecture design: 有关软件体系架构的设计→ high-level design
 - 包括：系统架构
 - 数据库->在设计类的基础上创建
 - interface
- Interface design:接口设计(不同层次的接口，包括UI) → high-level design
- Component design:组件设计（单个类的设计，同样以分析类图为基础）→ detailed design

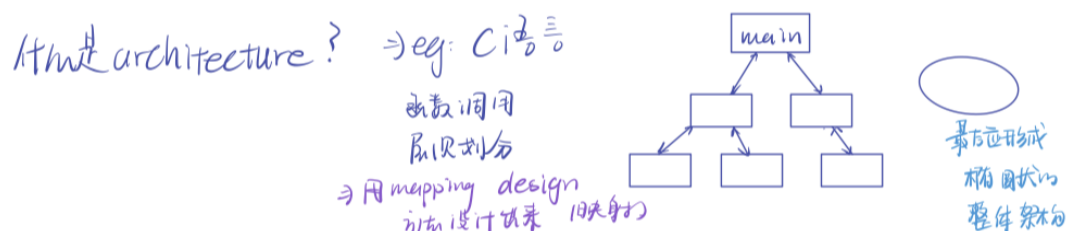


12.1 以软件工程上下文(Context)为依据进行设计

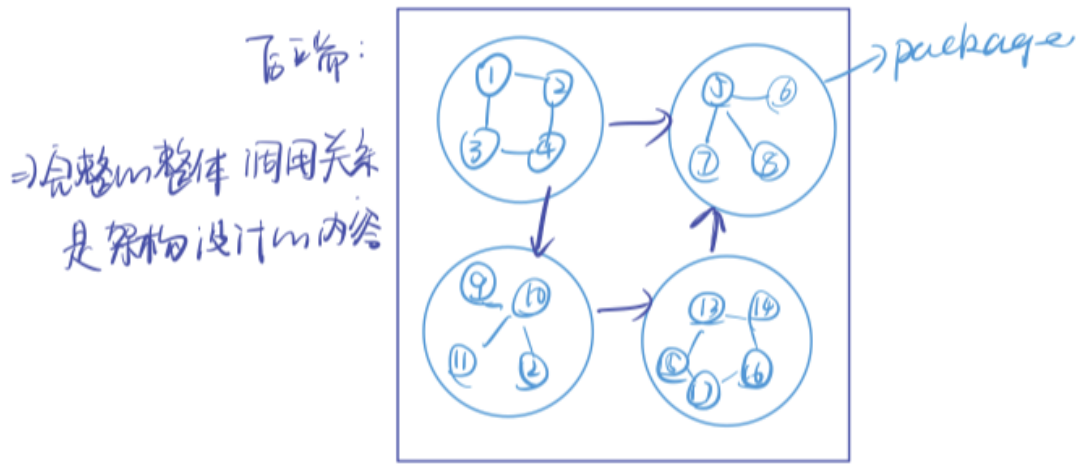
在完成需求建模后，Design是Modeling这个Activity最后的Action

架构举例：

1. C语言(函数调用)



2. 面向对象开发的体系架构



每个 package 内部m类之间有调用(接口)
 package 之间 有调用(接口)
 ⇒ 接口设计m internal interface m 设计.

※ 老师在黑板上画的软件体系架构图



※ 将需求模型翻译成设计模型 requirement model → design model

interface design: 需要Use Case的场景元素来搭建界面，所以依赖于场景元素里面的场景（不含功能模块），然后，类不仅有类内部的交互，还可能用到其他类的方法，而时序图描述的是一个Use Case中不同类的交互（比如Actor和Entity等），所以这里还需要时序图，故需要行为模型

data/class design: 包括数据库，数据结构，类设计，数据字典等。很显然需要Class Base Element中的类去搭建（和数据建模相关）其中 1. 数据库设计以分析类图为依据，一对一多对多等关系构建ER图进而建库；2. 数据结构设计也要参考类设计；3. 类的设计自然需要类图，同时完善协作关系；

Arch design: 这里需要用到类和类之间的协作关系，通过CRC卡片和类去划分功能相关的子系统，同时写子系统内部的类和方法（内部完善）进而再通过协作关系确定子系统间的通信方式，并根据这个写接口类，完成软件体系架构设计

组件设计: 1 需要参考分析类图，因为需要设计类之间的相互调用，通过看调用关系增加方法；2. 要看状态图和时序图，根据不同状态调用不同方法。

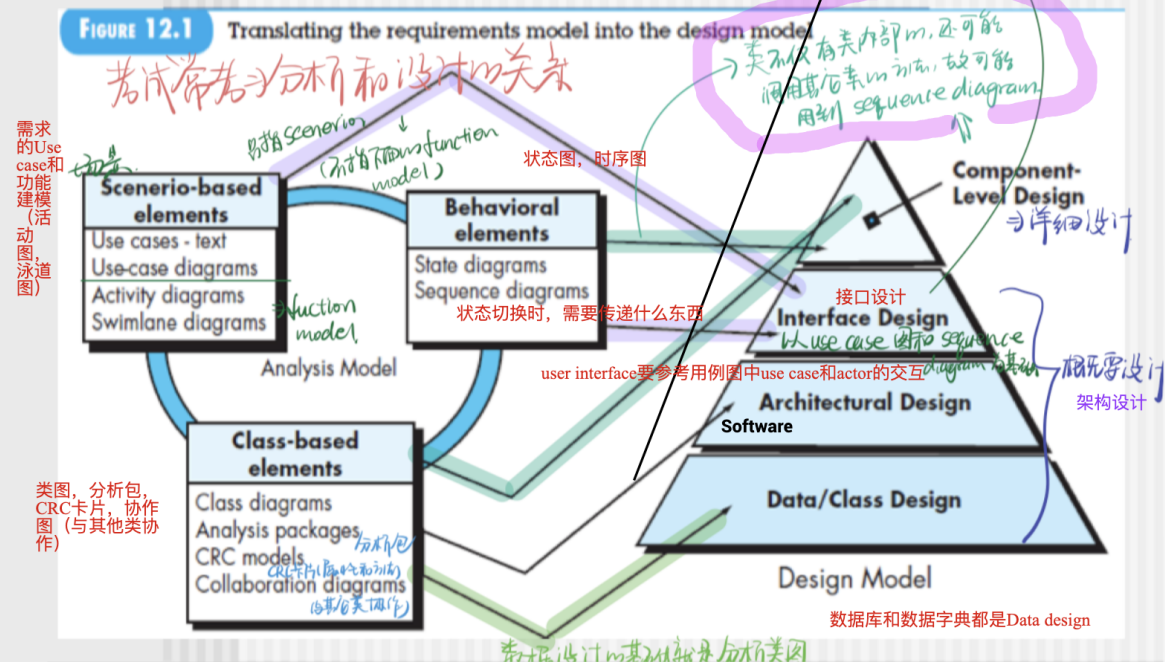
架构设计 Architectural Design 包括:
Interface Design (user, external, internal)
Software Architectural Design
Data/Class Design

基于场景的功能元素; ——界面设计
基于类的元素; ——类/数据设计, 体系结构设计, 界面设计
行为元素——界面设计, 组件级别设计

软件体系结构设计依据: 以类图和类之间的协作关系为重要依据, 设计时基于类图和CRC卡片把功能相关的构建子系统, 与系统内部的类与方法, 并明确子系统之间的通信方式, 进而增加接口类, 最终完成软件体系结构设计(必考)

接口: 包括 user interface (图形界面)
external interface (和外部的接口, 比如和支付宝支付模块的接口)
internal interface (子系统中内部接口, 如前端到后端)

Chapter 12 DESIGN CONCEPTS



※ 数据设计中的数据字典



12.2 设计过程 Design Process

设计的描述: 对需求阶段的结果不断进行迭代优化

随着设计迭代的发生, 随后的细化 (refinement) 导致了在更低的抽象层次上的设计表现。这些仍然可以追溯到需求, 但这种联系更加微妙

12.2.1 Software Quality Guidelines and Attributes

评估一个设计是否好的3个评估标准:

- Requirements** 设计应满足需求模型中的显示要求以及隐式需求, 即以分析建模 analysis modeling 为基础
The design should implement all of the explicit requirements contained in the requirements model, and it must accommodate all of the implicit requirements desired by stakeholders
需求包括: 显式需求, 隐式需求; 设计时 stakeholder 要参与
- Readable** 设计要好理解
- Picture** 设计要把前阶段的数据建模、功能建模、行为建模等完整表示出来

(TODO)

质量指导方针 Quality Guidelines (8个, 可能考选择题)

1. 完整地继承分析阶段的体系架构，并由recognizable的架构风格或模式构造而成(architectural styles or patterns)
2. 设计得出的结构应该
 1. 由好的组件构成composed of components
 2. 可以以进化方式evolutionary fashion实现，从而促进实施implementation和测试testing
3. 设计的结果应该模块化(modular)，构成子系统(subsystems)
4. 设计的数据结构，接口，组件设计必须清晰(distinct representation)
5. 设计应该易于实现，类的设计选用合适的数据结构(lead to data structures that are appropriate for the classes to be implemented and are drawn from recognizable data patterns)
6. 要高内聚,有独立的功能(components should exhibit independent functional characteristic)
7. 接口要尽量简单（比如参数少）（interface that reduce the complexity)
8. 设计的结果应可复用（using a repeatable method)
9. 用于表示设计的记号要有意义(notations that effectively communicates its meaning)

比如：命名应该有意义，便于理解

这些设计方针Design Guidelines是通过应用基本设计原则，系统方法论和彻底审查来实现的。

质量属性Quality Attributes ★【会考】

所有软件设计的目标——FURPS quality attributes

- F: **Functionality** 功能性 (方法功能是否满足要求)

评估方式：程序的函数集和函数function set & function，被delivered的函数的通用性generality以及整个系统的安全性security。

- U: **Usability** 可用性

评估方式：overall的美观性aesthetics，一致性consistency，导航navigation，文档documentation等

- R: **Reliability** 可靠性

评价方式：测量失败的频率frequency和严重程度severity，输出结果的精度accuracy(如：要求保留两位但只达到一位)，[平均故障时间](#)（MTTF, mean -time-to-failure），从故障中恢复的能力recover ability，以及程序的可预测性predictability。

- P: **Performance** 性能

评价方式：处理速度processing speed，响应时间response time，资源消耗consumption，吞吐量throughput和效率efficiency



- S: **Supportability** 可支持性

评价指标：可维护性(maintainability)【包括：可扩展性extensibility(系统要+功能)，适应性adaptability（可以在不同设备上适应)和可维护性serviceability（出了问题或变更需求如何定位可否修复)】。另外，可测试性testability，兼容性compatibility，可配置性configurability，可以安装系统的简化性ease，以及有哪些问题可以本地化(the ease with

which a system can be installed, and the ease with which problems can be localized)

上述质量属性一般同等重要，但在一些应用中可能会强调其中一些

12.2.2 软件设计的演化The Evolution of Software Design【不用看】

method:

- top-down structured method 自上而下的结构(C语言架构)
- object-oriented approach 面向对象设计方法
- aspect-oriented methods
- model-driven development
- test-driven development

这些method的共同特征:

1. M将需求模型转换为设计表示的机制mechanism
2. N表示功能组件及其接口的记号notation
3. H改进和分区的启发性(heuristics for refinement and partitioning)
4. G软件评估的指导方针(guidelines for quality assessment)

12.3 设计概念Design Concepts

12.3.1 抽象Abstraction

建模一直在抽象，代码是最终的抽象

Give an example for the abstraction

12.3.2 架构Architectural【这里往下一部分老师好像都没讲】

举例：前后端分离的架构

什么是架构：

architecture is the structure or organization of program components (modules), the manner in which these components interact, and the structure of data that are used by the components

即： 1. 对组件模块的组织； 2. 组件的交互方式； 3. 组件使用数据的结构

12.3.3 模式Patterns

解决某问题的具体步骤(成熟的方法)

a set of architectural patterns enables a software engineer to reuse design-level concepts.

12.3.4 分支Separation of Concerns

包括：

- modularity模块化（大项目分解为子系统，子系统分解为模块）
- aspect每个模块负责的事情
- functional independence功能独立
- refinement功能细化

12.18老师举例： $P1=c(P1)$ $P2=c(P2)$ $C(P1+P2)>C(p1)+C(p2)$ 体现拆分可以简化

12.3.5 模块化Modularity【看图】

Pi: 某功能

C: 复杂度

E: 工作量

一般对任务\$P1\$, \$P2\$, 如果\$C(P1)>C(P2)\$, 那么\$E(P1)>E(P2)\$

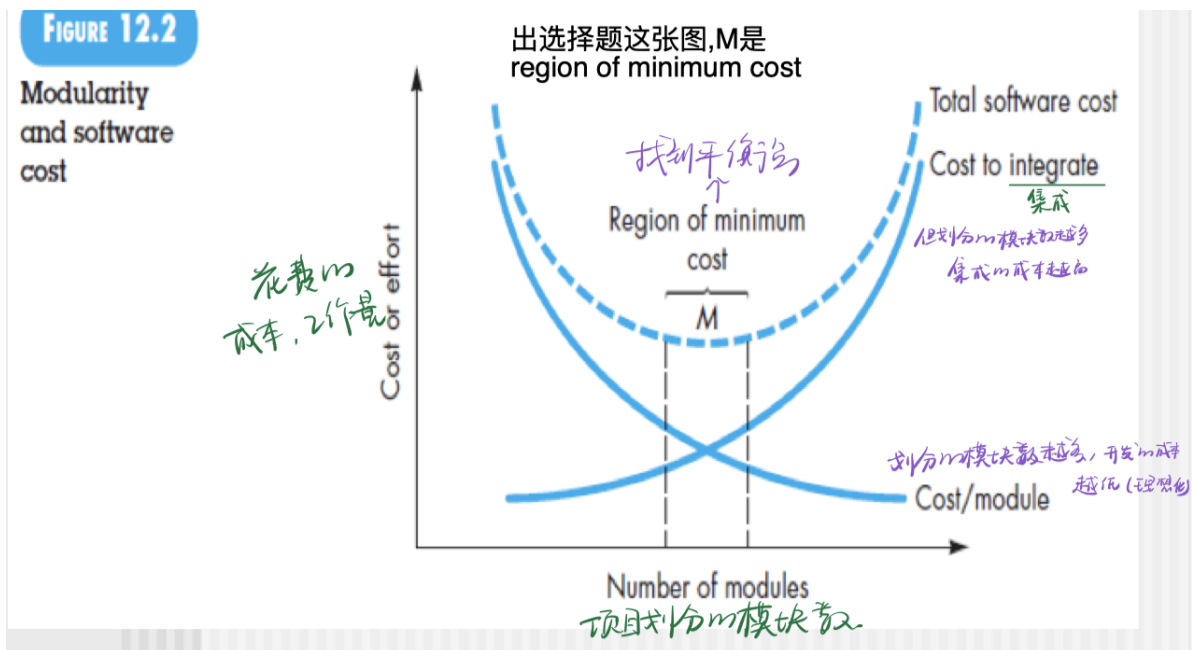
另外有:

$E(P1+P2) > E(P1) + E(P2)$

$C(P1+P2) > C(P1) + C(P2)$

分而治之在这里体现, 合在一起做工作量更大, 分而治之可以节约成本

※模块化和软件开销的关系



随着模块数量的增多, 用于集成integrate的开销越大, 而每个模块开发的成本平摊越少, 两者加起来类似一个二次函数, 这个二次函数有个最优区间, 叫最小开销区间 (M)

12.3.6 模块化中的信息隐藏Modularity→Info Hiding

12.18: C1 C2两个类彼此不知道对方类内的信息, 通过接口调用来获取要达到信息隐藏必须有功能独立性也就是模块化

应指定和设计模块, 以便模块中包含的信息 (算法和数据) 无法对不需要此类信息的其他模块无法访问
B调用A时, 有时需要A部分信息对B隐藏

12.3.7 功能的独立性 Functional Independence

separation of concerns, modularity, abstraction, information hiding 最终共同作用形成 Function Independence

独立性的两个标准: 内聚性和耦合性(cohesion和coupling)——高内聚低耦合

12.8 协作就是类之间的communication/coupling

12.3.8 细化Refinement 笔记上有细说【必考】

细化举例：

- 分析类细化到设计类（有的类会被合并或者拆分掉
 - 类的拆分
 - 类的合并
 - 方法优化（函数名、函数体、函数的参数、功能说明
 - 属性的规范
- 分析建模时画的sequence diagram
 - 接口设计的依据就是细化后的sequence diagram
- 设计类图中的每个方法需要绘制activity diagram
 - 依据activity diagram对功能进行细化

12.3.9 面向方面的设计(Design) aspects【不用看】

一个aspect是一个crosscutting concern的展示（WTF? ），它的实现方式为一个单独的module(component)

要跨不同的类进行交互（比如：interface class可能是个aspect）

12.3.10 重构Refactoring

重构的目的：避免代码冗余，调整不合适的数据结构等

[代码重构的方式\(Chapter 7\)](#)

12.3.11 面向对象的设计理念Object-Oriented Design Concepts

12.3.12 设计类Design Classes

【可能出考题：用做过的java项目解释5种设计类】

分析类转设计类，设计类的抽象等级要相对高些

※设计类的种类（5种，分别代表设计结构design architecture的不同layer）：

1. [Business domain classes](#) 业务类（由entity class转过来，基本是一一对一，可能有优化比如一个entity类分成2个类等）
2. [User Interface classes](#) 用户界面类的设计（由boundary class转过来，sequence diagram & use case diagram）
3. [Process classes/Control classes](#) 控制类（由，controller class转过来，往往没有属性，只有方法）对系统的可维护性很重要
4. [Persistent classes](#) 持久类（用于数据持久化，持久化类的实例对象将保存在数据库或文件中。人话版：用于访问数据库的class，存一些select/insert的方法，可以通过改变参数来使用相应的方法）
5. [System classes](#) 系统类（如Java的主类有main()方法）

※体现设计类好不好的4个指标

1. [Complete and Sufficient](#) 完整充分（属性方法的完整封装encapsulation）封装良好
2. [Primitiveness](#) 原始性（同样的功能和属性不能在2个或多个类中出现）不重复，每个属性方法都是原始的
3. [High Cohesion](#) 高内聚（属性和方法与类本身相关）
4. [Low Coupling](#) 低耦合（交互接口要简单）

低耦合体现在哪？【可能会考】

- 1.类的方法之间的协作中**参数**数量尽可能少，值和类型尽可能简单
- 2.类内的方法之间的**访问**可以尽可能频繁，类间和子系统间的访问要尽可能少，子系统要更少

接口类的存在就是为了低耦合；一个类更多应该访问邻界类，尽量少访问其他package或 subsystem；子系统中的一个类不需要知道其他类太多信息，最好通过内部接口或外部接口这些控制接口类去访问得到

